Introducción al Aprendizaje Automático y Compensadores Lineales

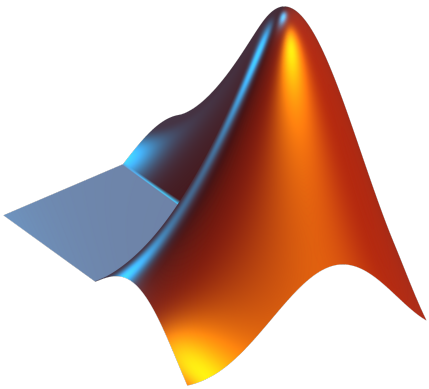
Fabrizio Benavidez[1]

https://github.com/Fabopy/Matlab-curso-

[1] *Facultad de Ciencias Exactas, Físicas y Naturales, Ingeniería Aeronáutica*

Resumen El siguiente trabajo consiste en una introducción al *Machine Learning*, empleando regresión lineal y redes neuronales. Además se emplean herramientas en MATLAB para la regresión a distintos sistemas.
También se emplean compensadores lineales para una respuesta deseada en un sistema.

Palabras claves: MATLAB, Compensadores, Aprendizaje, regresión, lineales



26 de octubre de 2026

Semana 10

[1] fabrizio.benavidez@mi.unc.edu.ar

Guía 19: Introducción al Aprendizaje Automático

Ejemplo 1: un simple problema de regresión

El siguiente código en MATLAB representa un conjunto de datos sobre el cancer y la cantidad de cigarrillos que se fuma por día. Se realiza una regresión lineal para predecir la mejor forma que se adapte a los datos (minimize los errores) y además una regresión de grado 15. En la figura 1 se grafica los resultados

```
clc
clear all
close all

%% Ejemplo 1
cig = 0:1:32;
cancer = 1.7*cig + 22 + 5*randn(size(cig));

fig = figure;
theme(fig, 'light')
%Casos de cancer
plot(cig, cancer, 'b')
hold on;

%Hacemos una regresion lineal
lin = polyfit(cig, cancer, 1);
cancer_est = polyval(lin, cig);
plot(cig, cancer_est, 'r')
xlabel('Cigarrillos por dia')
ylabel('Casos de cancer')

%Hacemos una regresion de grado 15
orden15 = polyfit(cig, cancer, 15);
cancer_est15 = polyval(orden15, cig);
plot(cig, cancer_est15, 'g')
```

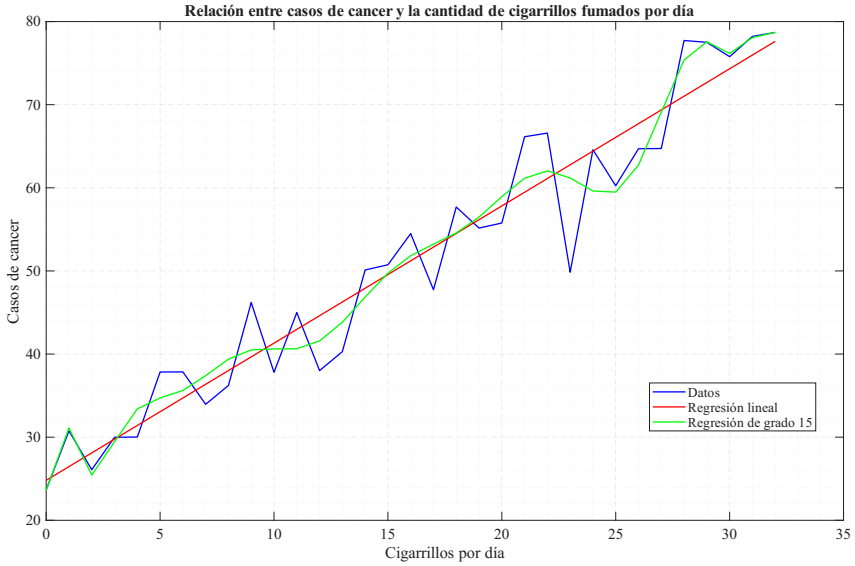


Figura 1. Gráfico de los datos

El modelo lineal tiene la ventaja computacional y que visualmente representa una función simple, marcando la tendencia general de los datos. En cambio, el polinomio de mayor grado representa la ventaja de que se ajustará más a los datos. Aunque parezca más razonable usar

un polinomio de mayor grado, este representa una clara desventaja. Según *Steven C. Chapra Raymond P. Canale : Método Numéricos para Ingenieros* (Capítulo 17, sección 17.2.1):

> Un problema potencial en la implementación de la regresión polinomial en la computadora es que las ecuaciones normales algunas veces están mal condicionadas. Esto se presenta especialmente cuando se planten polinomios de grado superior. En tales casos, los coeficientes calculados pueden ser altamente susceptibles al error de redondeo y, en consecuencia, los resultados serían inexactos.

Donde las ecuaciones normales son un conjunto de ecuaciones (que depende del grado del polinomio) lineales simultáneas. Por último, se observa que la variable `cancer` tiene un término random y en la regresión polinómica esto puede representar un problema causando el sobreajuste (*Overfitting*): condición donde en un modelo empieza a describir el ruido en lugar de la relación entre las variables. Este problema se puede ver en la gráfica de arriba.

Predicción de series temporales

La siguiente figura representa el precio de la soja y los modelos empleados para predecir el precio durante los próximos 3 años.

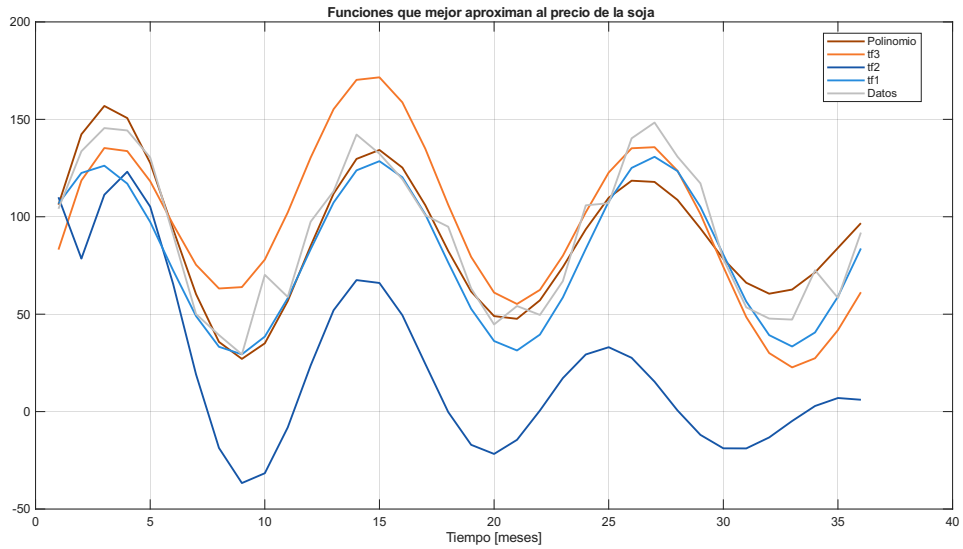


Figura 2. Modelos que mejor estiman el precio de la soja durante los próximos años

De los modelos empleados, el que mejor estima es el polinomio con un *Best Fit* de 63.59. Para tf1 58.21, tf2 -107.4 y tf3 35.67. Los modelos pueden superar el valor de 80 pero esto implica jugar con los distintos modelos hasta que encaje dentro de los valores de 80 (anteriormente realicé una prueba y obtuve un Best Fit de 82, pero al tener un término randn en la variable `y` éste varía con cada compilación que realice).

Para tf1, la función transferencia obtenida fue:

$$\frac{21{,}3s + 0{,}04801}{s^2 + 1{,}167 \times 10^{-9}s + 0{,}2707} \tag{1}$$

con 2 polos y un cero. Para tf2

$$\frac{14{,}3s^2 + 32{,}51s - 1{,}091}{s^3 + 2{,}413s^2 + 0{,}6228s + 0{,}823} \tag{2}$$

con 3 polos y 2 ceros. Para tf3

$$\frac{15{,}1s^3 + 49{,}35s^2 + 7{,}818s - 0{,}1673}{s^3 + 0{,}2033s^2 + 0{,}2728s + 0{,}05042} \tag{3}$$

con 3 polos y 3 ceros. Por último, el polinomio tiene la estructura $A(z)y(t) = B(z)u(t)+e(t)$, donde

$$A(z) = 1 - 0{,}6494\frac{1}{z} - 0{,}4911\frac{1}{z^2} + 0{,}5377\frac{1}{z^3} + 0{,}2277\frac{1}{z^4} \tag{4}$$

$$B(z) = 17{,}81\frac{1}{z} - 17{,}78\frac{1}{z^4} \tag{5}$$

Se observa que a medida que el sistema es más complejo (de mayor orden) pueden resultar peores que los sistemas menos complejos, tal como sucede con tf1 y tf3.

Encontrar padrones con redes neuronales

Veamos la siguiente figura

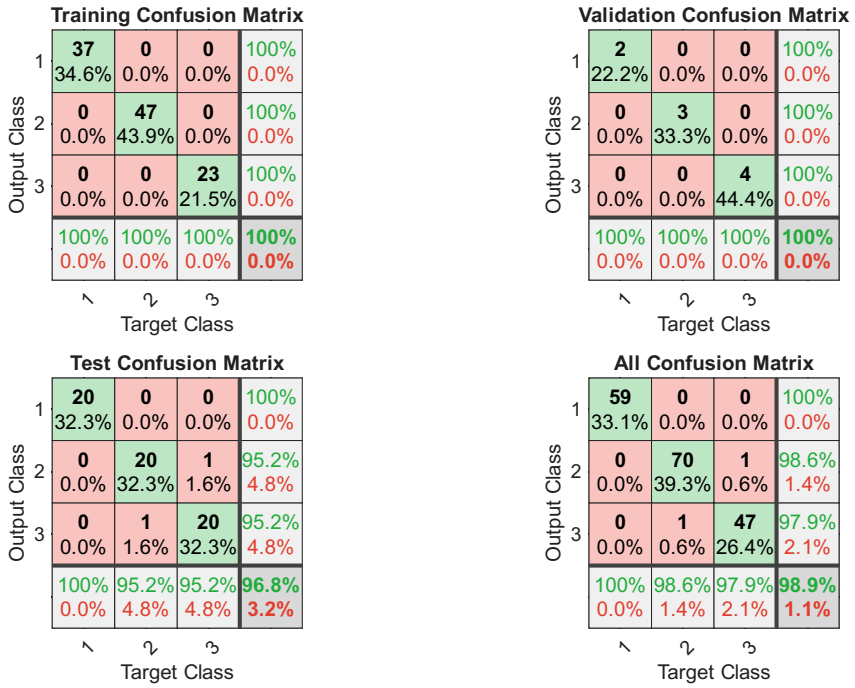


Figura 3. Plot confusion

Training Confusion Matrix

Corresponde a los datos utilizados para ajustar los pesos y sesgos de la red, es decir, para que aprenda a clasificar. En la imagen figura que hubo 37 aciertos para la clase 1, 47 para la clase 2 y 23 de la clase 3, lo que equivale al 100 % de clasificación en las muestras.

Validation Confusion Matrix

El conjunto de validación se utiliza para supervisar el aprendizaje y ayudar a decidir cuándo detenerlo, evitando seguir ajustado la red cuando empeora su comportamiento fuera del conjunto de entrenamiento. Dicha matriz depende de la cantidad de datos y cómo estos influyen sobre el porcentaje de aciertos.

Test Confusion Matrix

Evalúa el comportamiento de la red con datos que no se utilizaron para ajustar sus parámetros ni para decidir cuándo detener el entrenamiento. Es decir, la red reconoció 62, donde 2 de ellos (números fuera de la diagonal principal) son errores.

All Confusion Matrix

Reúne los resultados de entrenamiento, validación y prueba. El 98.9 % incluye las muestras de entrenamiento.

Por último, aumentar o disminuir la cantidad de neuronas ocultas no garantiza mejores predicciones (sobreajuste).

Guía 20: Compensadores Lineales

Veamos los siguientes sistemas:

$$G_1(s) = \frac{34}{s^3 + 11s^2 + 44s + 34} \quad G_2(s) = \frac{s^3 + 2s^2 + s + 1}{s^4 + s^3 + 8s + 1} \tag{6}$$

$$G_3(s) = \frac{1}{s^4 + 15s^3 + 100s^2 + 250s} \tag{7}$$

El siguiente código en MATLAB representa estos 3 sistemas y determina si son estables o no mediante los polos:

```
clc
clear
close all


function estable(num, den)
    G = tf(num, den);
    polos = pole(G);
    parte_real = real(polos);
    if any(parte_real >= 0)
        disp('El sistema es inestable o criticamente estable')
    else
        disp('El sistema es estable')
    end
end

%Para G1
G1 = tf([34], [1, 11, 44, 34]);
estable([34], [1, 11, 44, 34])
>> El sistema es estable

%Para G2
G2 = tf([1, 2, 1, 1], [1, 1, 0, 8, 1]);
estable([1, 2, 1, 1], [1, 1, 0, 8, 1])
>> El sistema es inestable o criticamente estable

%Para G3
G3 = tf([1], [1, 15, 100, 250, 0]);
estable([1], [1, 15, 100, 250, 0])
>> El sistema es inestable o criticamente estable
```

Se observa que los sistemas G_2 y G_3 son inestables ya que presentan polos positivos o nulos. Para esto, podemos compensar el sistema mediante una constante k. Para esto, probaremos distintos valores de k y ver si compensan o no el sistema. En la tabla ... se observa si dicho valor de k es ideal para el sistema

	G_1	G_2	G_3	Ninguno
$k = 204$		✓	✓	
$k = 23$		✓	✓	
$k = 0{,}01$	✓		✓	
$k = 0{,}8$	✓		✓	
$k = 5{,}5$	✓	✓	✓	

Aunque el cuadro represente cuáles valores de k estabilizan el sistema, el criterio es buscar los polos más a la izquierda y con menor parte imaginaria. Por lo tanto, para G_1

es recomendable $k = 0{,}8$; G_2 es recomendable $k = 5{,}5$ y por último G_3 es recomendable $k = 204$.

El siguiente código representa cómo se realizó esto en MATLAB

```
K = [204, 23, 0.01, 0.8, 5.5];
F = [G1, G2, G3];

j = 0;
for f = F
    j = j + 1;
    fprintf('G%g\n', j)
    for k = K
        fprintf('--------------------\n')
        fprintf('k = %g\n', k)
        sis = feedback(f, k);
        polos_sis = pole(sis);
        real_sis = real(polos_sis);
        fprintf('Polos %g\n', real_sis)
        if any(real_sis >= 0)
            continue;
        else
            fprintf('k = %g es ideal para G%g\n', k, j)
        end
    end
end
```

Veamos el siguiente sistema:

$$G(s) = \frac{4}{s(s+2)} \tag{8}$$

Su lugar de raíces y su respuesta escalón está en la figura 4

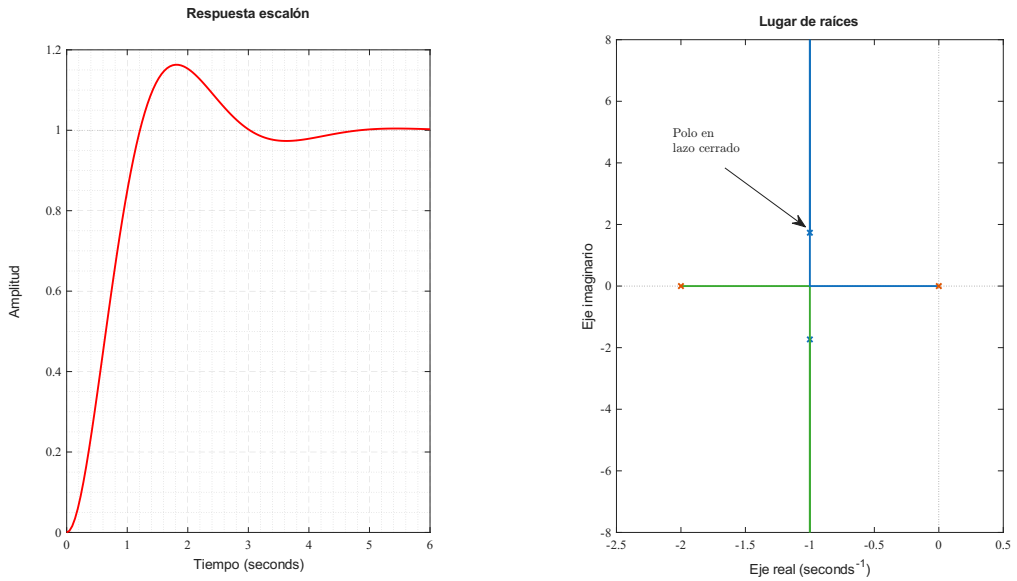


Figura 4. Respuesta del sistema $G(s)$ ante una entrada escalón y su lugar de raíces. Se observa que tiene polos en 0 y -2 y $-1 \pm 1{,}732i$ para el lazo cerrado.

Ahora, por alguna razón que desconocemos, queremos ubicar los polos en $-2 \pm 2\sqrt{3}i$, por lo que necesitamos un compensador. Para esto emplearemos la siguiente lógica para encontrar un compensador que pase por esos polos.

Compensador para $G(s)$

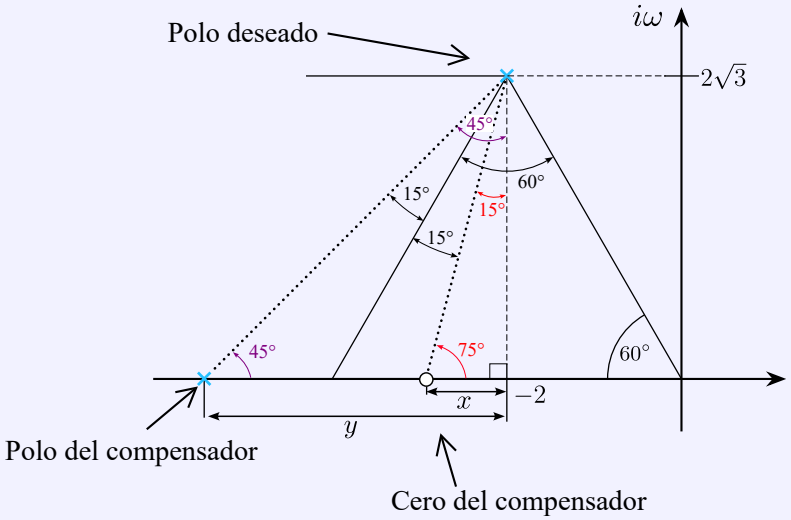
Se empleará el cálculo que propone *Katsuhiko Ogata : Ingeniería de Control Moderna*, capítulo 6, sección 6.6. Usaremos un compensador de adelanto. Un compensador de adelanto $H(s)$ tiene la siguiente función de transferencia:

$$H(s) = K_c \frac{s + \frac{1}{T}}{s + \frac{1}{\alpha T}} \tag{9}$$

Para determinar estos valores, determinemos la suma de los ángulos en la localización deseada de uno de los polos dominantes en lazo cerrado con los polos y ceros en lazo abierto del sistema original, determinando el ángulo ϕ necesario que va a contribuir al compensador. Por lo tanto:

$$\phi = 180° - 120° - 90° = -30° \tag{10}$$

El déficit en el ángulo de 30° debe estar contribuido por un compensador en adelante. Hay varias maneras de obtener la localización del cero y del polo del compensador en adelante. Veamos el siguiente método y para eso analicemos la siguiente figura:



Dado que el ángulo fue de 30°, entonces ese ángulo se lo divide por 2 y se obtiene (en la bisectriz) dos ángulos de 15°. Para calcular x e y podemos usar la siguiente expresión (se deriva de la figura por trigonometría):

$$s = \frac{2\sqrt{3}}{\tan \alpha} \tag{11}$$

Donde $\alpha = 75°$ y $45°$. Por lo tanto:

$$x = \frac{2\sqrt{3}}{\tan 75} = 0{,}9282$$

$$y = \frac{2\sqrt{3}}{\tan 45} = 3{,}4641$$

Si ahora a ambas expresiones le sumamos 2 obtenemos el cero y el polo del compensador:

$$c = x + 2 = 2{,}9282$$

$$p = y + 2 = 5{,}4641$$

Cabe aclarar que ambos resultados es el valor absoluto. Al estar en el semiplano izquierdo, c y p tienen signo negativo.
Para calcular K_c se determina a partir de la condición de magnitud:

$$\left| K_c \frac{s + 2{,}9282}{s + 5{,}4641} \frac{4}{s(s+2)} \right|_{s=-2+2\sqrt{3}i} = 1 \tag{12}$$

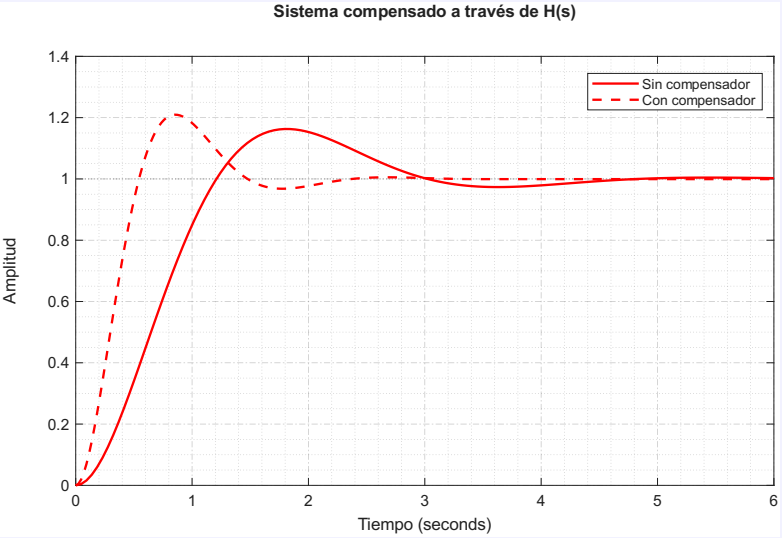
Despejando K_c y reemplazando en s por el valor, se obtiene:

$$K_c = 4{,}732$$

Por lo tanto, el compensador es:

$$H(s) = 4{,}732 \ \frac{s + 2{,}9282}{s + 5{,}4641} \tag{13}$$

La siguiente figura representa la respuesta del sistema compensado ante una entrada escalón



Código usado

```
%El problema sugiere un compensador en serie con realimentacion
    unitaria
%{
Segun Ogata, la adicion de ceros al sistema hace que el
lugar de raices se desplace a la izquierda, siendo
mas estable el sistema. En cambio, la adicion de polos
lo hace mas inestable (tiende el lugar de raices
a la derecha)
%}
Gs = zpk([], [0, -2], 4);
sist = feedback(Gs, 1);
fig = figure;
theme(fig, 'light');
subplot(1,2,1)
stepplot(sist, 'r')
```

```
subplot(1,2,2)
rlocus(sist)
hold on
rlocus(Gs)

%Compensador calculado siguiendo la bibliografia del Ogata
Hs = zpk([-2.9282], [-5.4641], 4.732);
Gs_compensado = feedback(Hs*Gs, 1);
stepplot(sist, 'r')
hold on
stepplot(Gs_compensado, 'r--')
legend('Sin compensador', 'Con compensador')
```

Por último, se puede usar funciones como `pidtune` o `pidtool` para ver cuáles son las funciones compensadores para compensar un sistema $G(s)$. Por ejemplo, para el sistema $G_1(s)$ probamos un compensador proporcional integral y un PID (proporcional, integral y derivativo).

```
parametros_pi = pidtune(G1, 'pi');
>>
                1
  Kp + Ki * ---
                s

  with Kp = 1.85, Ki = 1.86

parametros_pid = pidtune(G1, 'pidf');
>>
                1              s
  Kp + Ki * --- + Kd * --------
                s           Tf*s+1

  with Kp = 1.99, Ki = 2.42, Kd = 0.348, Tf = 0.00494
```